

### 1.- sync.Mutex

Permite sincronizar el acceso a una sección crítica (por ejemplo, una variable compartida).

```
var mu sync.Mutex

mu.Lock()
// sección crítica
mu.Unlock()
```

### 2.- sync.RWMutex

Es como Mutex pero permite múltiples lectores simultáneamente si no hay escritores.

```
var rw sync.RWMutex

rw.RLock()
// sección de solo lectura
rw.RUnlock()

rw.Lock()
// sección de escritura
rw.Unlock()
```

### 3.- sync.WaitGroup

Sirve para esperar a que un grupo de goroutines terminen.

```
var wg sync.WaitGroup

wg.Add(1)
go func() {
    defer wg.Done()
    // trabajo concurrente
}()

wg.Wait()
```

#### 4.- sync.Once

Ejecuta una función solo una vez, sin importar cuántas veces se invoque.

```
var once sync.Once

once.Do(func() {
    fmt.Println("Esto se ejecuta solo una vez")
})
```

#### 5.- sync.Cond

Proporciona una forma de esperar hasta que ocurra una condición.

```
var mu sync.Mutex
cond := sync.NewCond(&mu)

go func() {
    mu.Lock()
    cond.Wait() // espera hasta que otro despierte
    fmt.Println("Despertado")
    mu.Unlock()
}()

mu.Lock()
cond.Signal() // o cond.Broadcast()
mu.Unlock()
```

## Ejercicios

### Ejercicio 1: Contador seguro con sync.Mutex

Implementa un contador concurrente al que varias goroutines incrementan sin condiciones de carrera.

```
1  package main
2
3  import (
4      "fmt"
5      "sync"
6  )
7
8  func main() {
9      var mu sync.Mutex
10     counter := 0
11     var wg sync.WaitGroup
12
13     for i := 0; i < 1000; i++ {
14         wg.Add(1)
15         go func() {
16             defer wg.Done()
17             mu.Lock()
18             counter++
19             mu.Unlock()
20         }()
21     }
22
23     wg.Wait()
24     fmt.Println("Resultado esperado: 1000, obtenido:", counter)
25 }
26
```

## Ejercicio 2: Lectores y escritores con sync.RWMutex

Simula un recurso compartido que se lee y escribe concurrentemente.

```
1  package main
2
3  import (
4      "fmt"
5      "sync"
6      "time"
7  )
8
9  type SafeMap struct {
10     m map[string]string
11     rw sync.RWMutex
12 }
13
14 func main() {
15     sm := SafeMap{m: make(map[string]string)}
16
17     sm.m["clave"] = "valor"
18
19     // Lector
20     go func() {
21         sm.rw.RLock()
22         fmt.Println("Leyendo:", sm.m["clave"])
23         sm.rw.RUnlock()
24     }()
25
26     // Escritor
27     go func() {
28         sm.rw.Lock()
29         sm.m["clave"] = "nuevo valor"
30         sm.rw.Unlock()
31     }()
32
33     time.Sleep(time.Second)
34 }
35
```

## Ejercicio 3: Esperar múltiples goroutines con sync.WaitGroup

```
1  package main
2
3  import (
4      "fmt"
5      "sync"
6  )
7
8  func worker(id int, wg *sync.WaitGroup) {
9      defer wg.Done()
10     fmt.Printf("Trabajador %d terminado\n", id)
11 }
12
13 func main() {
14     var wg sync.WaitGroup
15     for i := 1; i <= 5; i++ {
16         wg.Add(1)
17         go worker(i, &wg)
18     }
19     wg.Wait()
20     fmt.Println("Todos los trabajadores han terminado")
21 }
22
```

## Ejercicio 4: Ejecutar solo una vez con sync.Once

```
1  package main
2
3  import (
4      "fmt"
5      "sync"
6  )
7
8  func main() {
9      var once sync.Once
10
11      f := func() {
12          fmt.Println("Se ejecuta una sola vez")
13      }
14
15      for i := 0; i < 10; i++ {
16          go once.Do(f)
17      }
18
19
20      // Esperamos un poco para que todas las goroutines terminen
21      time.Sleep(time.Millisecond * 1000)
22      var wg sync.WaitGroup
23      wg.Add(1)
24      go func() {
25          defer wg.Done()
26      }()
27      wg.Wait()
28  }
29
```

## Ejercicio 5: Comunicación controlada con sync.Cond

```
1  package main
2
3  import (
4      "fmt"
5      "sync"
6      "time"
7  )
8
9  func main() {
10     var mu sync.Mutex
11     cond := sync.NewCond(&mu)
12     ready := false
13
14     go func() {
15         mu.Lock()
16         for !ready {
17             cond.Wait()
18         }
19         fmt.Println("La condición se cumplió")
20         mu.Unlock()
21     }()
22
23     // Simular alguna preparación
24     mu.Lock()
25     ready = true
26     cond.Signal()
27     mu.Unlock()
28     //esperar
29     time.Sleep(time.Millisecond * 1000)
30 }
31
```